

SecureShift Backend System Architecture

Lou Best

May 2026

1. Introduction

1.1 Purpose

This document provides an overview of the SecureShift backend architecture, including system structure, access control design, data ownership rules, and current implementation gaps. It is intended to support development, onboarding, and future system evolution.

1.2 Scope

This document focuses on the backend API (app-backend), including its interaction with the Employer Panel and Guard App. It covers architecture, security model, data relationships, and key subsystems.

1.3 Intended Audience

- Backend developers
 - Frontend developers integrating with the API
 - Project mentors and assessors
-

2 System Context

2.1 Platform Overview

SecureShift is a multi-client workforce management platform designed for the security industry. It supports employer-driven shift creation and management, guard onboarding and verification, and workforce coordination.

2.2 Main Application Areas

- Backend API (Node.js/Express)
- Employer Panel (React)
- Guard App (React Native / Expo)

2.3 Core Business Workflows

- Employer onboarding and profile management
 - Guard registration and verification
 - Shift creation and assignment
 - Guard application and attendance tracking
 - Payroll and reporting (partially implemented)
-

3. High-Level Architecture

3.1 Major Components

SecureShift consists of three primary application components and one shared persistence layer:

- **Backend API (`app-backend`)**
Node.js and Express service providing REST endpoints, authentication, RBAC, business logic, persistence, and integration with supporting sub-systems such as verification, uploads, and notifications.
- **Employer Panel (`app-frontend/employer-panel`)**
React-based web frontend used by employers and administrative users to manage profiles, shifts, sites, favourites, and other employer-facing workflows.
- **Guard App (`guard_app`)**
Expo / React Native mobile application used by guards to manage profile details, view and apply for shifts, complete attendance workflows, and interact with notifications.
- **MongoDB Database**
Primary persistence layer used by the backend through Mongoose models for users, shifts, attendance, verification, messaging, and related platform data.

3.2 Component Interaction Summary

The backend acts as the central coordination layer between client applications and the database. Both the Employer Panel and Guard App consume backend REST endpoints exposed under `/api/v1`. The backend is responsible for enforcing authentication, role-based access control, business rules, and persistence.

At a high level:

- the Employer Panel communicates with the backend for employer and admin workflows
- the Guard App communicates with the backend for guard-facing workflows
- the backend reads from and writes to MongoDB

- supporting subsystems such as verification, uploads, and notifications are coordinated through backend modules

3.3 Request and Data Flow

A typical request follows this path:

Client -> Route -> Middleware -> Controller -> Service/Adapter -> Model -> Database

In the current codebase, many endpoints still follow a simpler flow:

Client -> Route -> Middleware -> Controller -> Model -> Database

This reflects the current architecture state, where some areas such as payroll, fatigue monitoring, guard scoring, documents, and verification adapters have clearer separation into service or adapter layers. Other parts of the backend, including shift creation, attendance, branch/site logic, availability, and incident workflows, remain more controller-driven and are candidates for future service-layer consolidation.

4. Repository and Codebase Structure

4.1 Monorepo Structure

The SecureShift project is organised as a monorepo containing backend, frontend, and mobile application components.

SecureShift/ |- app-backend/ |- app-frontend/ |- guard_app/ ‘- docker-compose.yml

This structure enables coordinated development across multiple clients while maintaining a shared backend API and data model.

4.2 Backend Source Layout

The backend is structured into modular directories under **app-backend/src**:

- **routes** — API endpoint definitions and middleware composition
- **controllers** — request handling and orchestration logic
- **services** — business logic (currently partial implementation)
- **models** — Mongoose schemas and database interaction

- **middleware** — authentication, RBAC, logging, validation
 - **adapters** — external integrations and subsystem logic
 - **utils** — reusable helper functions
-

4.3 Documentation Strategy

Documentation is maintained alongside the codebase under `app-backend/docs`.

Current documentation includes:

- RBAC design (`rbac.md`)
- verification subsystem (`verification.md`)
- architecture documentation (this document)

The strategy is to maintain documentation as a living reference aligned with the current codebase, rather than static diagrams that quickly become outdated.

5. Backend Architectural Style

5.1 Current Request Flow

The backend follows a modular Express architecture where requests pass through:

Route -> Middleware -> Controller -> Model -> Database

Middleware layers handle authentication, RBAC, and logging before requests reach controllers.

5.2 Current Layer Responsibilities

- **Routes**
 - Define endpoint paths and attach middleware
 - Apply authentication and role guards
 - In some areas, routes also contain inline authorization helpers
- **Middleware**
 - Authentication using JWT
 - User loading where full user records are required
 - Role-based and permission-based access control
 - Logging, audit middleware, and error handling
- **Controllers**
 - Handle request/response lifecycle
 - Coordinate application logic

- In several modules, controllers still contain business rules directly
 - **Services**
 - Contain reusable business logic where service-layer separation has been introduced
 - Current examples include payroll, fatigue monitoring, guard scoring, documents, and verification adapters
 - **Models**
 - Define Mongoose schemas, validation rules, indexes, virtual fields, and persistence relationships
-

5.3 Current vs Target Layering

Current State: - Many endpoints still use controller-driven logic directly - Service-layer abstraction exists in selected areas, including payroll, fatigue monitoring, guard scoring, documents, and verification adapters - Some route files use inline role guards rather than the central RBAC middleware - Attendance check-in/check-out logic remains close to the controller/model layer

Target State: - Controllers remain thin - Business logic is moved into reusable service layers - Models remain responsible for persistence and validation

This transition is planned incrementally rather than through large-scale refactoring. ***

6. Access Control and Security Model

6.1 Authentication

Authentication is implemented using JWT tokens validated through the `auth` middleware.

Authenticated requests pass through the `auth` middleware. Some routes also use `loadUser` where the controller requires the full user document rather than only the JWT payload.

Swagger documentation uses bearer authentication for API testing.

6.2 Current RBAC Implementation

RBAC is implemented through middleware using role-based and permission-based checks.

However, RBAC is currently **partially consolidated**, with multiple approaches present across the codebase.

6.3 Implemented Roles vs Documented Roles

Currently implemented roles: - admin - employer - guard

Documented or partially implemented roles: - super_admin - branch_admin - client

These additional roles exist in documentation and middleware but are not consistently supported across the full system.

The `User.role` enum defined in the user schema is currently the authoritative list of roles that can be persisted for user accounts.

A current implementation risk is role-definition drift. For example, some routes and middleware reference expanded roles such as `super_admin`, `branch_admin`, `client`, or `employee`, but the persisted `User.role` enum currently only supports `guard`, `employer`, and `admin`. This means route-level role checks must be reviewed against the schema before being treated as reliable access-control guarantees.

Middleware references to roles such as `super_admin`, `branch_admin`, and `client` represent planned or partially implemented functionality unless fully supported by the schema and route access rules.

6.4 Access Control Principles

Access control is designed to prioritise **data safety and ownership boundaries**.

Routes must not grant broad access unless: - the underlying controller or service enforces ownership filtering

Example: - `/api/v1/users/guards` is intended to be restricted because it exposes global guard data - however, the current route references `ROLES.ADMIN` and `ROLES.EMPLOYEE`, where `ROLES.EMPLOYEE` does not align with the persisted user-role enum - this indicates role-definition drift and reinforces the need for route-level RBAC review

6.5 Security Notes and Limitations

- RBAC patterns are not yet fully standardised
- Some endpoints rely on role checks without consistent data filtering
- Future improvements should unify RBAC enforcement and tenant-aware access rules

7. Data Ownership and Tenant Scoping

7.1 Multi-Tenant and Marketplace Context

SecureShift operates as a multi-client workforce platform where multiple employers use the same backend and database.

The current implementation is best understood as a hybrid model:

- employers own branches/sites through `employerId`
- employers create shifts through `createdBy`
- guards are global users rather than employer-owned records
- guards connect to employers through shift applications, assignments, attendance, payroll, and related workflows

This means employer-facing queries must be scoped carefully. In a marketplace-style model, employers should not receive broad global guard data. Instead, employer access to guard information should generally be based on shift-specific relationships such as applicants, assigned guards, payroll records, or attendance records connected to shifts owned by that employer.

7.2 Current Ownership Signals

The current data model includes partial ownership indicators:

- `User.role` identifies whether an account is a guard, employer, or admin
 - `Branch/Site` includes `employerId`, identifying the employer that owns the site
 - `Shift` includes `createdBy`, identifying the employer/admin user that created the shift
 - `Shift.siteId` links a shift to an employer-owned branch/site
 - `Shift.applicants` records guards who have applied for a shift
 - `Shift.acceptedBy` records the approved/assigned guard
 - `Shift.guardIds` supports preselected or assigned guard references
 - `Guard` users appear globally accessible unless scoped through shift, payroll, attendance, or other relationship data
 - No explicit `Company` model exists
-

7.3 Current Scoping Gaps

Tenant boundaries are not consistently enforced across all endpoints.

In particular: - guard data - shift data - payroll-related data
may not be fully scoped to employer ownership.

7.4 Ownership Rules

The system should enforce the following principle:

All employer-facing queries must be scoped by ownership fields such as `employerId`, `createdBy`, or equivalent relationships.

Employers must never receive unscoped global data.

7.5 Practical Implication

Recent backend changes (e.g. `/users/guards`) highlight the need for:

- explicit ownership filtering
- consistent tenant-aware query design

This will guide future backend improvements.

Examples of endpoints and workflows that require ownership-aware review include:

- `GET /api/v1/users/guards`
- `GET /api/v1/users/guards/:id/score`
- `GET /api/v1/users/favourites`
- `GET /api/v1/availability/:userId`
- `GET /api/v1/attendance/:userId`
- document administration routes accessible by employer roles
- notification creation and broadcast routes
- payroll and reporting endpoints

These endpoints must be reviewed to ensure they enforce proper tenant scoping before broader role access is enabled.

8. Domain Model and Relationships

8.1 Core Entities

The SecureShift backend is built around a set of core domain entities representing users, workforce operations, and system interactions.

- **User**
 - Base account model for all roles
 - Stores authentication and shared profile data
- **Guard**
 - Extends user role
 - Represents workforce members
 - Includes verification status, licence data, and shift participation
- **Employer**

- Extends user role
 - Represents organisations managing workforce operations
 - Owns branches/sites and creates shifts
 - **Shift**
 - Represents work assignments created by employers
 - Includes scheduling, assignment, and lifecycle state
 - **Branch/Site**
 - Represents employer-owned locations
 - Intended to scope shifts and workforce operations
 - **ShiftAttendance**
 - Tracks guard attendance and check-in/check-out events
 - **Availability**
 - Stores guard availability for shift matching
 - **Message**
 - Represents communication between users
 - **Verification Models**
 - GuardVerification
 - ManualVerification
 - **EOI (Expression of Interest)**
 - Represents pre-onboarding company interest
 - **Payroll**
 - Stores payroll records derived from completed shifts and attendance data
 - Supports approval, processing, and export workflows
 - **Incident**
 - Records guard-related incident data
 - Used by guard scoring and operational risk workflows
 - **Notification**
 - Represents system/user notifications
 - **Equipment**
 - Represents equipment-related records
 - **Role**
 - Represents role/permission structures where implemented
 - **AuditLog**
 - Supports auditability of backend actions where logging is used
-

8.2 Relationship Summary

- A **User** is the base entity for all roles
- A **Guard** participates in shifts and has attendance records
- An **Employer** owns branches/sites and creates shifts
- A **Shift** is created by an employer and may be linked to a branch/site
- A **ShiftAttendance** record links a guard to a shift
- A **Branch/Site** belongs to an employer

- Verification records are linked to guards
-

8.3 Known Model Ambiguities

Several relationships are not yet fully defined or enforced:

- Guards appear globally accessible rather than employer-scoped
- Shifts do not consistently reference `employerId` or `branchId`
- No explicit `Company` model exists
- Ownership boundaries rely partially on `createdBy` rather than explicit fields

These ambiguities contribute to challenges in enforcing tenant-level data isolation.

9. API Surface and Functional Areas

9.1 Endpoint Groupings

The currently mounted backend route groups, as defined in `src/routes/index.js`, are:

- `/api/v1/documents`
- `/api/v1/health`
- `/api/v1/auth`
- `/api/v1/shifts`
- `/api/v1/messages`
- `/api/v1/admin`
- `/api/v1/availability`
- `/api/v1/users`
- `/api/v1/rbac`
- `/api/v1/branch`
- `/api/v1/attendance`
- `/api/v1/incidents`
- `/api/v1/notifications`
- `/api/v1/payroll`
- `/api/v1/equipment`

These mounted route groups should be treated as the source of truth for currently reachable backend API areas.

9.2 Versioning and Swagger

All endpoints are versioned under:

/api/v1

Swagger UI is used as the primary interface for: - API discovery - testing endpoints - validating request/response structures

9.3 Notes on Route Coverage

Swagger is useful for API discovery and testing, but mounted route files should be treated as the source of truth for currently reachable endpoints.

Because Swagger can scan route files that exist in the repository, it may document endpoints from files that are not currently mounted under /api/v1. This creates a possible documentation drift risk where Swagger appears to expose an endpoint that is not actually reachable through the Express route index.

For this reason, `src/routes/index.js` should be checked when confirming whether an API group is currently exposed.

9.4 RBAC Route Considerations

The `/api/v1/rbac` route group is currently mounted, but the associated route file appears to include middleware-related logic and may not represent a clean, stable RBAC management API.

This suggests that the RBAC route implementation requires review before being treated as a production-ready endpoint group.

10. Recent Backend Improvements and Architecture Impact

Recent backend work has improved the maintainability, reliability, and future-readiness of the SecureShift backend.

Improvement	Architecture impact
Payroll service and cleanup	Moves payroll calculation, date-range handling, role scoping, approval, processing, and export logic into a dedicated service layer. This improves maintainability and prepares payroll data for reporting and analytics.

Improvement	Architecture impact
Fatigue monitoring service	Introduces reusable workload-safety logic for assessing guard fatigue based on shifts per week, hours per day, and hours per week. This supports safer shift assignment and creates a foundation for future guard recommendation logic.
Guard scoring service	Combines shift completion, punctuality, and incident data into a reusable scoring calculation. This shows how attendance, shift, and incident data can support future operational intelligence.
Assigned guard and applicant relationships	Shift records now more clearly represent applicants, accepted guards, assigned guard virtuals, and guard references. This supports the marketplace-style workflow where guards apply to shifts and employers approve assignments.
Attendance validation	Attendance check-in/check-out includes location validation, assigned-guard checks, duplicate check-in prevention, and geofence-style radius checking. This logic is important for payroll reliability but remains a candidate for future service-layer consolidation.
RBAC and scoping review	Analysis of guard access, role drift, and employer visibility has clarified the need for route-level RBAC cleanup and marketplace-aware data scoping.
Architecture documentation	This document records the backend structure, risks, ownership model, and roadmap to support maintainability, onboarding, and future development.

11. Current Gaps, Risks, and Architectural Debt

11.1 RBAC Inconsistencies

- Multiple RBAC patterns exist across middleware and routes
 - Role definitions are not fully aligned between documentation and schema
 - Some endpoints rely on role checks without sufficient data scoping
-

11.2 Tenant Scoping Risks

- Employer-facing endpoints are not consistently scoped
 - Some queries currently return global data without ownership filtering
 - Example:
 - `/api/v1/users/guards` exposes global guard data and requires RBAC and marketplace-aware scoping review
-

11.3 Service Layer Gaps

- Business logic is inconsistently distributed
 - Controllers often contain logic that should be moved to services
 - Limited reuse of business rules across modules
-

11.4 Documentation and Implementation Drift

- Documentation may not reflect current API paths or schema details
 - Some workflows (e.g. verification) are partially implemented
 - Swagger and code should remain the primary sources of truth
-

11.5 Operational and Environment Issues

- Local environment setup inconsistencies
- Docker build issues in frontend
- Port conflicts during development

These issues do not prevent system operation but introduce friction and inconsistency in development workflows.

11.6 Mounted Route and Swagger Drift

Swagger may show endpoints from route files that exist in the repository even where those routes are not mounted through `src/routes/index.js`. This can

make API documentation appear broader than the actually reachable API surface.

Recommendation: treat `src/routes/index.js` as the source of truth for exposed route groups and periodically reconcile Swagger documentation with mounted routes.

11.7 Attendance and Site Geolocation Mismatch

Attendance check-in validates the guard location against `shift.siteId.location.latitude` and `shift.siteId.location.longitude`. However, the `Branch` model currently stores address-style location fields such as line, city, state, postcode, and country. This creates a possible mismatch between the attendance geofence logic and the site schema.

Recommendation: standardise branch/site geolocation fields or ensure shift-level latitude/longitude is consistently used for attendance validation.

12. Recommendations and Roadmap

12.1 Short-Term

- Reconcile mounted route groups with Swagger documentation
 - Fix role-definition drift, including unsupported role constants such as `employee`
 - Review `/users/guards` and guard score visibility against marketplace access rules
 - Standardise branch/site geolocation fields used by attendance validation
-

12.2 Mid-Term

- Introduce shift-specific applicant access, such as `GET /api/v1/shifts/:shiftId/applicants`
 - Continue moving attendance, shift approval, branch/site, and availability logic into services
 - Consolidate route-level authorization into shared RBAC middleware rather than inline route guards
 - Add service-level and endpoint-level regression tests for shift, attendance, payroll, and RBAC workflows
-

12.3 Long-Term

- Develop operational analytics using attendance, payroll, shift, incident, and guard score data
- Support employer labour cost summaries, guard earnings summaries, projected versus actual labour cost, and reliability metrics
- Use fatigue, availability, attendance, and guard score data as foundations for future AI-assisted guard recommendation